

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA



Tecnologias Para o Desenvolvimento Web

Raimundo Marcos Gonçalves Ludgério
Professor

E-mail: raimundo.marcos@ifpb.edu.br

Conteúdo programático

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

- Revisão HTML/CSS;
- Document Object Model;
- Introdução a JavaScript;
- Princípios Básicos da Linguagem;
- Variáveis e constantes;
- Tipos Primitivos;
- Operadores;
- Condicionais;
- Funções.

REVISANDO...

HTML (HyperText Markup Language)



- Estruturar o conteúdo da página
- Organizar o layout básico (sem estilos visuais)
- Inserção de mídia
- Navegação
- Formulários e inputs de dados
- Acessibilidade e semântica

CSS (Cascading Style Sheets)



- Estilização de elementos HTML
- Layout e posicionamento
- Responsividade
- Animações e transições
- Personalização visual com classes e IDs
- Organização e manutenção visual

DOM (Document Object Model)

Uma interface de programação que representa um documento

Uma árvore de objetos para manipulação por linguagens como JavaScript

DOM (Document Object Model)

Criação da Árvore: Quando um navegador carrega uma página web, ele processa o HTML e o converte numa estrutura em árvore, onde cada parte do documento (como um elemento, texto ou comentário) é um nó.

DOM (Document Object Model)



Representação de Objetos: A DOM representar esses elementos como objetos com propriedades e métodos.

Acesso e Manipulação

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O código executado no navegador (tipicamente JavaScript) pode usar a DOM para:

- Alterar o conteúdo: Modificar textos ou imagens.
- Modificar o estilo: Adicionar ou remover classes CSS

Acesso e Manipulação

O código executado no navegador (tipicamente JavaScript) pode usar a DOM para:

- Reagir a eventos: Adicionar "ouvintes" de eventos (como cliques) para disparar ações quando um evento ocorre.

JAVASCRIPT

JavaScript (JS)

- Manipulação do DOM
- Interatividade
- Validação de formulários
- Requisições assíncronas (AJAX/fetch)
- Armazenamento local de dados
- Criação de componentes dinâmicos
- Integrações com APIs e bibliotecas

COMO ADICIONAR JAVASCRIPT



- Utilizando a tag **<script>**
- Utilizando arquivo externo com extensão js
- No final do **<body> (melhora a performance)**

COMO ADICIONAR JAVASCRIPT


```
<body>
    <script>
        console.log("Olá, mmundo!")
    </script>
</body>
```

Princípios Básicos da Linguagem

VARIÁVEIS E CONSTANTES

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Existem três maneiras de declarar variáveis:

- **let:** variável de escopo
- **const:** constantes
- **var:** variáveis globais

Obs: O **;** no JavaScript é opcional.



Tipos Primitivos

- String: Sequência de texto
- Number: Valores numéricos
- Boolean: Verdadeiro ou falso
- Array: Estrutura que permite armazenar vários valores em um único referência
- Object: Tudo em JavaScript é um objeto e pode ser armazenados em uma variável



COMENTÁRIOS

- /* */ - Comentário em bloco
- // - Comentário em linha



Operadores

Um operator é um **símbolo matemático** que produz um **resultado** baseado em dois valores (ou variáveis).

Operadores

Operador	Detalhes	Símbolo	Exemplo
Adição	Adicionar dois números. Concatenar duas Strings	+	5 + 6; “Olá” + “Mundo”;
Subtração, multiplicação e divisão	Exatamente a mesma coisa da matemática	-, *, /	9 - 3; 8 * 2; 15 / 5;
Atribuição	Guardar um valor e associar a uma referência	=	let nome = “Raimundo”;
Igualdade Restrita	Verifica se dois valores são iguais.	===	let num = 3; 3 === 4; (falso)
Negação	Utilizado para negar um valor booleano	!	!(3===4);
Diferente	Compara se dois valores não são iguais	!==	let nome = “a”; nome !== “b”;

Operadores

Operador	Detalhes	Símbolo	Exemplo
E lógico	Verdade quando as duas operações forem verdade	&&	true && true
OU lógico	Verdade quando pelo menos uma das operações forem verdades	||	false || true

Operadores

JavaScript permite usar operadores lógicos com valores que não são booleanos.

O comportamento é baseado na ideia de valores true e false.

```
console.log(null || "valor padrão"); // "valor padrão"
console.log("texto" && 123);         // 123
console.log(0 && "oi");              // 0 (false)
console.log(!0);                     // true
```



Condicionais

Condicionais são estruturas de código usadas para testar se uma expressão retorna verdadeira ou não.

Uma forma muito comum de condicionais é a instrução **if...else**.

Condicionais


```
<input type="text" name="nome" id="nome" placeholder="Informe o seu nome">
<script>
    let nome = document.getElementById("nome").value;
    if (nome === "Raimundo") window.alert("Olá, Raimundo. Você é o administrdor");
    else window.alert("Verifique os dados e tente novamente! ");
</script>
```

Switch/case

Para condições simples, o JavaScript contém a estrutura Switch/case.

```
let cor = "vermelho";
switch (cor) {
    case "azul":
        console.log("Cor do céu");
        break;
    case "vermelho":
        console.log("Cor do fogo");
        break;
    default:
        console.log("Outra cor");
}
```



LAÇOS

Laço for:

```
for (let i = 0; i < 5; i++) {
    console.log("Número: " + i);
}
```

LAÇOS

Laço while:

```
let contador = 0;
while (contador < 3) {
  console.log("Repetição " + contador);
  contador++;
}
```



LAÇOS

Laço do…while:

```
let i = 0;
do {
    console.log("Número:", i);
    i++;
} while (i < 5);
```

Funções

Functions são uma forma de empacotar a funcionalidade que são reutilizáveis.

É possível definir um bloco de código como uma função que é executada quando é chamado pelo nome da função.

FUNÇÕES

- **getElementById():** Retorna o elemento HTML com o ID especificado
- **getElementsByClassName():** Retorna uma coleção de elementos que possuem a classe especificada
- **getElementsByTagName():** Retorna uma coleção de todos os elementos com a tag HTML especificada (por exemplo, div, p, a)

FUNÇÕES

- **querySelector():** Retorna o elemento no documento que corresponde ao seletor fornecido.
- **querySelectorAll():** Retorna uma NodeList (uma coleção de nós) de todos os elementos no documento que correspondem ao seletor fornecido.

FUNÇÕES


```
// Seleciona o primeiro parágrafo com a classe .paragrafo
let primeiroParagrafo = document.querySelector('.paragrafo');
console.log(primeiroParagrafo.textContent);


// Seleciona o elemento com o id 'titulo'
let titulo = document.querySelector('#titulo');
console.log(titulo.textContent);

// Seleciona o primeiro <span>
let span = document.querySelector('span');
console.log(span.textContent);
```

FUNÇÕES


```
let titulo = document.getElementById('titulo');
console.log(titulo.textContent);

let paragrafos = document.getElementsByClassName('paragrafo');
console.log(paragrafos[0].textContent); // Parágrafo 1
console.log(paragrafos[1].textContent); // Parágrafo 2

let spans = document.getElementsByTagName('span');
console.log(spans[0].textContent); // Texto dentro do span
```

ARGUMENTOS

Os argumentos ficam dentro dos parênteses, separados por vírgulas se houver mais de um argumento.

```
function multiplicacao(num1, num2) {
    let result = num1 * num2;
    return result;
}
multiplicacao(8,2)
```

ARGUMENTOS OPCIONAIS


```
function multiplicacaoOpc(num1 = 0, num2 = 1) {
    let result = num1 * num2;
    return result;
}

console.log(multiplicacaoOpc(8))
```

VARIÁVEIS FUNÇÕES

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O JS permite armazenar funções em variáveis

```
const minhaFun = function (numero) {
    return numero * 3
}

console.log(minhaFun(8))
```

Funções Anônimas

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Funções anônimas são funções que não possuem um nome **identificador**.

Elas são declaradas sem um nome e geralmente são usadas em contextos onde a função é usada como valor ou passada como argumento para outras funções.

Funções Anônimas



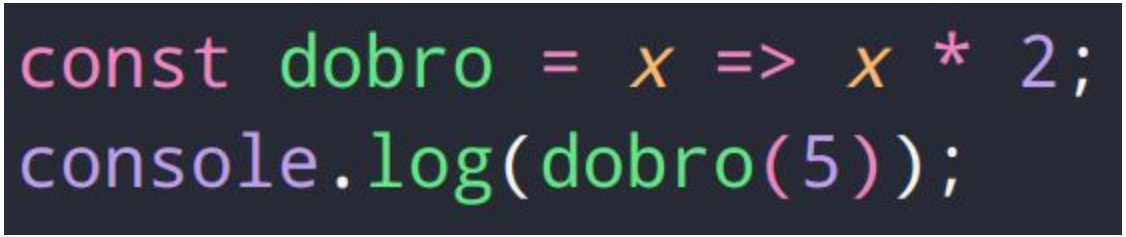
- Não possuem nome: São definidas sem um identificador
- São expressões: Podem ser atribuídas a variáveis ou passadas como argumentos
- Úteis para callbacks: Muito usadas em eventos e operações assíncronas

Funções Anônimas


```
const funAnonima = () => {
    console.log("Essa é uma arrow function")
}

array.map(function(item) {
    return item * 2;
})
```

Funções Anônimas



Função anônima auto-executável (IIFE)

Uma IIFE (“Immediately Invoked Function Expression”, ou “Expressão de Função Invocada Imediatamente”) é um padrão em JavaScript onde você define uma função e logo em seguida a executa automaticamente — sem precisar chamá-la depois usando nome.

Função anônima auto-executável (IIFE)


```
(function () {
    console.log('Esta função executa imediatamente!');
})();
```

```
// Com parâmetros
(function (mensagem) {
    console.log(mensagem);
})('Hello World!');
```



Eventos

São estruturas de código que detectam atividades no navegador e executam o código em resposta.

exemplo é lidar com o evento de **click**, que é acionado pelo navegador quando o usuário clica em algo com o mouse.

Eventos


```
<!DOCTYPE html>
<html lang="pt-br">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Introdução a Javascript</title>
</head>

<body>
    <script>
        document.querySelector("html").addEventListener("click", function () {
            alert("Ai! Pare de me cutucar!");
        });
    </script>
</body>
</html>
```

Eventos em botões e formulários

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O JavaScript permite interagir dinamicamente com elementos da DOM.

Portanto, é possível adicionar eventos a botões, textos, mensagens e etc.

Eventos


```
<body>
    <button id="meuBotao">Clique aqui</button>
    <input type="text" id="meuInput" placeholder="Digite algo">
    <div id="info"></div>
    <script>
        document.getElementById("meuBotao").addEventListener("click", () => {
            let valor = document.getElementById("meuInput").value;
            document.getElementById("info").innerHTML = valor;
        });
    </script>
</body>
```

Orientação a Objetos



Um objeto é uma coleção de dados e/ou funcionalidades relacionadas (que geralmente consistem em diversas variáveis e funções – que são chamadas de propriedades e métodos quando estão dentro de objetos).

Orientação a Objetos

O JavaScript também implementa o paradigma orientado a objetos.

Portanto, é possível construir sistemas trabalhando com os pilares da orientação a objetos: herança, polimorfismo, encapsulamento e abstração.

Orientação a Objetos

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Pode-se utilizar os objetos de maneira tradicional:

```
export default class Pessoa {
    constructor(nome, idade) {
        this.nome = nome;
        this.idade = idade;
    }

    apresentar() {
        console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`);
    }
}
```

Orientação a Objetos

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Pode-se utilizar os objetos de maneira tradicional:

```
import Pessoa from "./Pessoa.js";

var p1 = new Pessoa();
p1.idade = 37;
p1.nome = "Maria";
p1.apresentar();
```

Orientação a Objetos

Podemos trabalhar com JSON (JavaScript Object Notation)



Orientação a Objetos


```
var pessoa = {
    nome: ["Bob", "Smith"],
    idade: 32,
    sexo: "masculino",
    interesses: ["música", "esquiar"],
    bio: function () {
        alert(
            this.nome[0] +
            " " +
            this.nome[1] +
            " tem " +
            this.idade +
            " anos de idade. Ele gosta de " +
            this.interesses[0] +
            " e " +
            this.interesses[1] +
            ".",
        );
    },
    saudacao: function () {
        alert("Oi! Eu sou " + this.nome[0] + ".");
    },
};
```

Orientação a Objetos


```
pessoa.nome || pessoa["nome"];
pessoa.nome[0];
pessoa.idade;
pessoa.interesses[1];
pessoa.bio();
pessoa.saudacao();
```

Métodos de arrays

ARRAY HELPERS (MÉTODOS DE ARRAY)

Eles permitem aplicar uma função a cada elemento (ou selecioná-los, ou “reduzi-los”) sem precisar escrever o laço for manualmente.

Eles retornam novos valores (ou um novo array), sem modificar o array original (caso o callback não o faça explicitamente).

ARRAY HELPERS (MÉTODOS DE ARRAY)


```
const pessoas = [
    { nome: "Alice", idade: 28, cidade: "São Paulo" },
    { nome: "Bruno", idade: 34, cidade: "Rio de Janeiro" },
    { nome: "Carla", idade: 22, cidade: "Belo Horizonte" },
    { nome: "Diego", idade: 40, cidade: "Curitiba" },
    { nome: "Eduarda", idade: 19, cidade: "Porto Alegre" },
    { nome: "Fábio", idade: 31, cidade: "Recife" },
    { nome: "Gabriela", idade: 27, cidade: "Fortaleza" },
    { nome: "Henrique", idade: 45, cidade: "Manaus" },
    { nome: "Isabela", idade: 23, cidade: "Salvador" },
    { nome: "João", idade: 37, cidade: "Brasília" }
];
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Vamos criar um programa que imprima cada pessoa:

```
for(p of pessoas){
    console.log(p["nome"]);
    console.log(p.idade)
}
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

Agora vamos selecionar apenas as pessoas que tem mais de 20 anos.

```
let pessoas_mais_20 = []
for(p of pessoas){
    if(p.idade >= 20) pessoas_mais_20.push(p)
}
for(p of pessoas_mais_20){
    console.log(p)
}
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

- Para esses cenários, o JS disponibiliza alguns métodos para trabalhar com arrays
- array.**forEach(function())** -> varre todos os elementos do array
- array.**find(function())** -> Retorna o primeiro elemento que satisfaz a condição de teste.

ARRAY HELPERS (MÉTODOS DE ARRAY)

- array.**some(function())** -> Verifica se pelo menos um elemento satisfaz a condição
- array.**every(function())** -> Verifica se todos os elementos satisfazem a condição.

ARRAY HELPERS (MÉTODOS DE ARRAY)

- array.**map(function())** -> percorre cada elemento do array, aplica uma função transformadora a ele, e retorna um novo array com os resultados
- array.**filter(function())** -> percorre cada elemento do array e retorna um novo array contendo somente os elementos para os quais o teste retornou true.

ARRAY HELPERS (MÉTODOS DE ARRAY)



- array.**reduce(function())** -> “reduz” o array em um único valor, aplicando repetidamente uma função acumuladora.



forEach()

- Não retorna nada (undefined)
- Não modifica o array original (a menos que você modifique dentro da função)
- Não pode ser interrompido com break ou return

forEach()


```
pessoas.forEach((pessoa, indice) => {
    console.log(`A pessoa ${pessoa.nome} está na posição ${indice}`)
})
```

```
pessoas.forEach((pessoa, indice) => console.log(`${pessoa.nome} - ${indice}`))
```

Varre todos os elementos.



FIND()

- Retorna o elemento encontrado
- Retorna undefined se nenhum elemento satisfazer a condição
- Para no primeiro elemento que satisfaz a condição

FIND()


```
let maioridade = pessoas.find(pessoa => pessoa.idade >= 18)
console.log(maioridade)
```

```
let maioridade = pessoas.find(function (pessoa) {
    return pessoa.idade >= 18;
})
console.log(maioridade)
```



some()

- Retorna boolean (true/false)
- Para na primeira ocorrência que satisfaz a condição
- Útil para validações

SOME()


```
const comecaComA = pessoas.some(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // true
```

```
const comecaComA = pessoas.some(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // true
```



EVERY()

- Retorna boolean (true/false)
- Para no primeiro elemento que não satisfaz a condição
- Array vazio retorna true (condição vacuamente verdadeira)

EVERY()


```
const comecaComA = pessoas.every(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // false
```

```
const comecaComA = pessoas.every(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // false
```



MAP()

- Percorre cada elemento do array
- Aplica uma função transformadora a ele
- Retorna um novo array com os resultados
- O array original **não é alterado** (a menos que você faça modificações explícitas nos objetos internos)
- Tamanho do novo array = tamanho do array original.

MAP()


```
const nome_pessoas = pessoas.map(function(pessoa){
    return pessoa.nome
});
console.log(nome_pessoas);
```

```
const nome_pessoas = pessoas.map(pessoa => pessoa.nome);
console.log(nome_pessoas);
```



FILTER()

- Percorre cada elemento do array
- Aplica uma função de teste (predicate) que retorna true ou false para cada elemento
- Retorna um novo array contendo somente os elementos para os quais o teste retornou true

FILTER()


```
const pessoas_de_jampa = pessoas.filter(p => p.cidade == "João Pessoa");
console.log(pessoas_de_jampa)
```

```
const pessoas_de_jampa = pessoas.filter(function (p) {
    return p.cidade == "João Pessoa"
});
console.log(pessoas_de_jampa)
```



REDUCE()

- “Reduz” o array em um único valor
- Aplicando repetidamente uma função acumuladora
- Pode somar, multiplicar, agrupar, contar, fazer transformações complexas
- A função de callback recebe dois argumentos principais: o acumulador e o valor atual.

REDUCE()


```
// contando quantas pessoas tem em cada cidade
const resultado = pessoas.reduce((contador, pessoa) => {
    if(!contador[pessoa.cidade]){
        contador[pessoa.cidade] = 0;
    }
    contador[pessoa.cidade]++;
    return contador
},{});
console.log(resultado)
```

ECMAScript

ECMASCRIPT



- Uma especificação (ou padrão), criada pela ECMA International, que define a sintaxe, tipos de dados, objetos, controle de fluxo, e outros comportamentos básicos de linguagens de script.
- JavaScript é uma **implementação** desse padrão.

Por que o ECMAScript foi criado?

Nos anos 90, com o surgimento do JavaScript no navegador Netscape, outras empresas começaram a criar suas próprias versões.

Para evitar incompatibilidades entre navegadores, foi criada uma padronização – o ECMAScript – para que todas as versões seguissem as mesmas regras.

Por que o ECMAScript foi criado?

- Garantir compatibilidade entre navegadores e plataformas.
- Permitir evolução controlada do JavaScript com novas versões (por exemplo, ES6, ES2020, etc.).
- Padronizar novas funcionalidades, como async/await, let/const, classes, módulos, entre outras.

ATIVIDADE

Jogo de Par ou Ímpar contra a Máquina

Criar uma página que permita jogar Par ou Ímpar contra a máquina.

- O usuário escolhe Par ou Ímpar
- O usuário digita um número
- A máquina escolhe um número aleatório entre 0 e 10
- Soma-se os dois números.
- Se o resultado for par e o usuário tiver escolhido Par, ele vence (e vice-versa).

Jogo de Par ou Ímpar contra a Máquina



- A atividade deve ser feita individualmente.
- Não é permitido consultar internet, IA, colegas ou materiais externos.
- É permitido apenas utilizar o editor de código e o navegador para testar o programa.
- A entrega deve conter um único arquivo HTML com o código JavaScript embutido.
- Ao final do tempo, o aluno deve entregar o arquivo conforme instrução do professor.

OBRIGADO!

Repositório com os códigos: https://github.com/raimundoludgerio/estrutura-de-dados/tree/main/web

MDN contributors. Getting Started JavaScript. 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Getting_started/Your_first_website/Adding_interactivity

Contato:

raimundo.marcos@ifpb.edu.br

As Dúvidas podem ser enviadas pelo e-mail, conversar pessoalmente após a aula ou em algum horário a combinar.